# Introduction to Textual Data Representation Methods and Word Embeddings

Navid Nobani

# Index of Contents

**1 Introduction: Human language** **2**

**2 What is word embedding?** **3**
2.1 Presentation Methods . . . . . 5
2.1.1 One-Hot Encoding . . . . . 5

**3 Word2Vec** **9**
3.1 Training . . . . . 10
3.2 Mechanism . . . . . 11
3.2.1 Dot-product . . . . . 13
3.2.2 Softmax Function . . . . . 15
3.3 Implementing Word2Vec in Python and R . . . . . 19
3.3.1 Python Implementation . . . . . 19
3.3.2 R Implementation . . . . . 20
3.3.3 General Considerations . . . . . 20

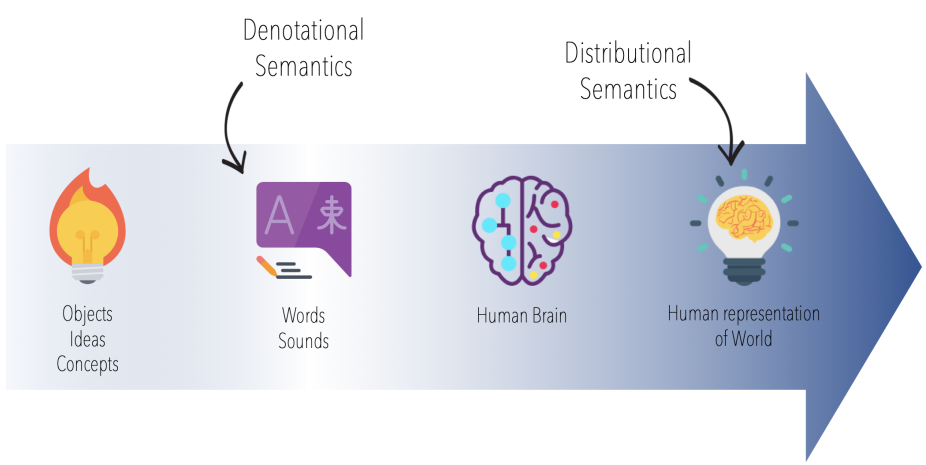
# 1 Introduction: Human language

Somewhere between 50,000 and 2 million years ago, humans started to interact with each other through language. The mystery surrounding the origin of oral communication has attracted much attention. Despite the flurry of hypotheses that have been formulated, questions such as 'Has language evolved gradually or suddenly?', 'what prompted the first human beings to link chains of sounds to objects in the real world?', 'When did grammar appear?' are probably bound to remain unanswered. What we know for sure, though, is that humans created "representations" in order to transfer ideas, concepts and meanings to other humans.

Of course, it has taken thousands of years for this process to come to an end, and during this period, the human brain was able to create a model of reality, to link words (symbols) not just assign them to a single concept/idea but to understand their dynamics with the other symbols That is why if someone should ask:

> Fetch me a pair of scissors to cut this rope!

if instead of scissors you spotted a knife, you would in all likelihood pick it up and hand it to the person who made the request. In fact, in the model of reality that lies in your brain, scissors are used to cut, and so is a knife. While at first glance this may seem trivial, trying to replicate this phenomenon outside the human brain is everything but simple!
Before delving deeper into the issue of which options are available, let's review how the human brain deals with words:




The above figure conveys the journey from the tangible to the conceptual in human language comprehension. It contrasts two perspectives on semantics—denotational, which links words to the world's objects and ideas, and distributional, which maps language to mental representations. The first view suggests language is a direct naming system for the physical and abstract elements of our environment. The second view proposes that our understanding of language is shaped by cognitive processes, resulting in an internal worldview constructed from linguistic encounters. Together, they offer a comprehensive picture of how language functions as a bridge between the external environment and our inner cognitive landscape.

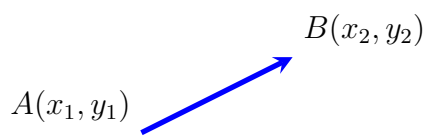
## 2 What is word embedding?

Imagine we wished to replicate the process we talked about in the introduction, using a computer algorithm, which takes concepts as input and outputs a model which represents the real world with it to realize the semantic similarities among words. We're lucky that we shouldn't start from the first step, mapping concepts/ideas to symbols. Human beings have done it for us for thousands of years!
Unfortunately though, while for performing a large variety of machine learning algorithms we can work with raw text, the moment we decide to go further and discover the link among words, their semantic similarity or cluster them by subject we need to take a further step and do something called word embedding. What is a word-embedding you ask? In a nutshell, it's a Vector! To be more precise, a dense numerical vector with a predefined length. Before going any further let's make sure what a vector is.

In algebra, a vector is a geometric object which provides two pieces of information about two hypothetical points A and B:

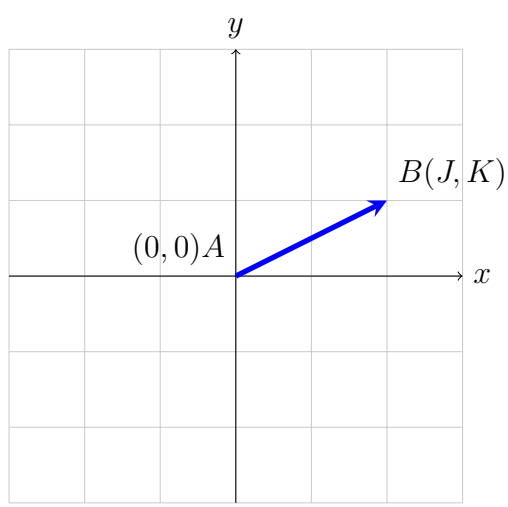
- Magnitude (How far A is from B)
- Direction (In which direction points are connected together)

$$v = \overrightarrow{AB}$$



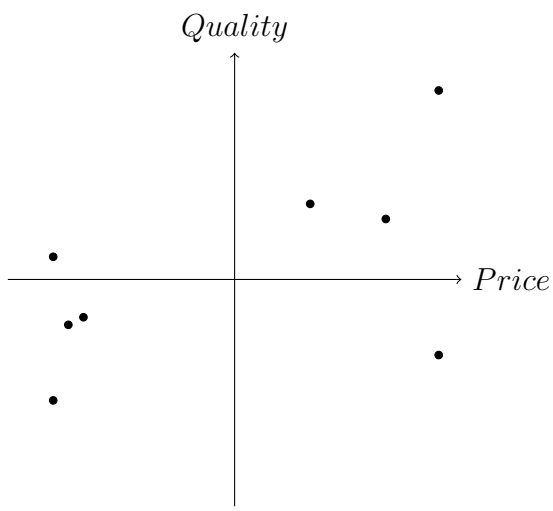

As you can see in the figure above, to describe a vector we need to have two points both having their own coordinates: $(x_1, y_1), (x_2, y_2)$. we can shift the point of origin and make it coincide with the origin. To do so the only thing we need to do is to subtract one of the points (Ex. $(x_1, y_1)$) from both points:

$$(x_1 - x_1, y_1 - y_1), (x_2 - x_1, y_2 - y_1) \Rightarrow (0, 0), (J, K)$$

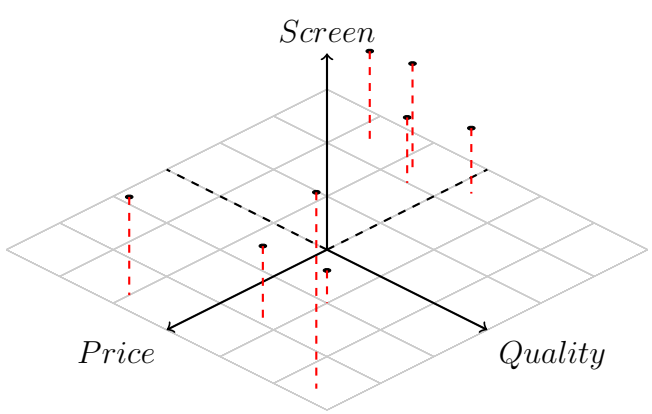


Now we can represent the vector just by one point: $(J, K)$ This brings us to the second definition of a vector, i.e. from a topology point of view: A vector is an element of a vector space $\mathbb{R}^n$ with n as a number of dimensions. (in our example above $n = 2$).

But, what does it mean exactly a point like $(J, K)$? Well, It's just a way to describe a point. Let's take a look at a simple example:
Imagine that you want to buy a phone and there are just two things which are important to you: Price and quality. For sure you'll have dozens of phones to choose from and therefore you need to compare them together. To do this, you assign a pair of $(price, quality)$ to each one. in this way you can create something like this:




Looking at the plot above we can several information one of which is similarity/dissimilarity among different phones. What we have just created is a simplified(we deliberately just used two features) representation of a complex reality.
In order to make our representation more accurate, let's add another feature: "screen" size. We will get something like this:




We see that by creating a simplified representation of a complex object like a cellphone we can infer various information from them.

Now let's try to do the same thing for words. How, in your opinion, we can do it? what would be our axes?

Let's review what we have done until now: We have taken some complex entities(cell phones) from a context (let's say the market), found some features which can describe the entities and in the end we were able to represent the entities using those features (Price, Quality and Screen size in our example). Saying so, we can see that following the same process, -at least in theory! - we should be able to do the same thing with words by performing the following steps:

- Find a context that contains entities
- Finding some entities we want to represent
- Finding some features to use for the presentation
- Mapping the initial entities in a vector space using the features

In our case, the context would be documents/corpus consisting of words. Entities are words (Ex. *cat*, *bat*, *today*, ...) and we just need to find some representative features. Well, this step may be harder than it looks! looking at our example words, In your opinion, what can we use as a feature? word length maybe? grammatical rule of words? (Noun, verb,...), maybe, the meaning of the word? You see that finding representative features for words is not so straightforward. One of the reasons for this is that despite their simple look, each word we use in our daily language is a result of a gradual evolution of human language which takes hundreds of thousands of years. It means words are not just simple symbols that indicate a unique concept/object, instead, they are highly complex entities which have multidimensional relationships with other entities. The key point, here, seems to have a way to map the words in a vector space but is it the blocking points? let's find it out! Considering words as independent symbols maybe the most simple way to show them in a vector space is to use the famous One-Hot Encoding.

## 2.1 Presentation Methods

Various encoding strategies and embedding techniques are employed to translate textual data into numerical vectors, which are essential for processing language within the realms of Natural Language Processing (NLP) and the broader spectrum of Machine Learning. Due to its pervasive application and foundational significance in these fields, it is necessary to delve into the intricacies of one such method—One-Hot Encoding. We will explore the mechanics of how One-Hot Encoding operates, its inherent simplicity in representing textual elements as binary vectors, and critically analyze its constraints.

### 2.1.1 One-Hot Encoding

One-Hot Encoding serves as a straightforward yet powerful approach, where each word in a given corpus is transformed into a unique vector. This vector corresponds to a dimension in the feature space that equals the size of the vocabulary. In this binary vector representation, a '1' is placed in the position denoted for the word, while all other positions are filled with '0's. This process effectively creates a distinct signature for each word but also leads to high-dimensional and sparse vectors.

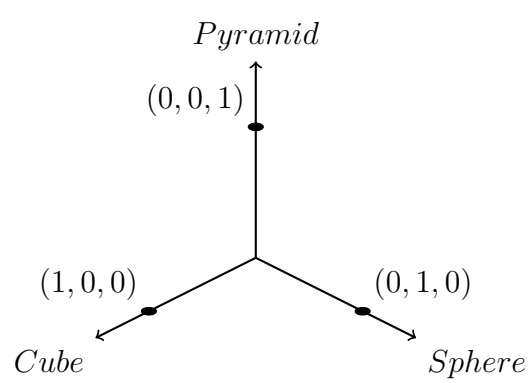
However, this method's limitations become apparent when dealing with large vocabularies, resulting in vectors with vast lengths and inefficient storage and processing requirements. Furthermore, One-Hot Encoding does not capture the semantic relationships between words—synonyms are as different as unrelated words in this representation, and the context of words within a sentence is entirely lost. Consequently, while One-Hot Encoding provides an entry point into textual data representation, it is often eschewed in favour of more sophisticated and semantically rich representations, such as embeddings, that aim to overcome these limitations. These advanced techniques not only reduce dimensionality but also strive to encode the contextual and semantic nuances inherent in natural language. To better understand its concept, let's go through an example:
Assume you have three completely different objects (Ex. a cube, a sphere and a pyramid). One way to address them is their names, symbols that represent the objects they are assigned to. But what if we want to represent them not by their symbols but by vectors? To do so all we need to do is first create an empty $(3 * 1)$ vector (because we have 3 objects):

$$Cube = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \; Sphere = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \; Pyramid = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

The role of these currently empty vectors is to describe each object so each row in the vector represents a feature, something we can assign to objects. In this case, since the objects are completely independent, these features are exactly the same as the objects themselves. (Consider these features as Cubeness, Pyramidness and Sphereness). As a result, each object can only and only have one of the features at the same time(sphere has sphereness and not cubeness) which means each vector will have exactly one 1 and 2, i.e. $(n - 1)$, zeros.

$$Cube = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \; Sphere = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \; Pyramid = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

We can even create a graphical representation of our vectors.




Well if it works for three words why it shouldn't work for all the words in the English language? Well, considering various tenses, counts and genders, there are about 13 million tokens in the English language! Even if it doesn't much sense, let's just for the sake of the argument go forward and try to replicate what we did for all the English tokens. Considering

the first word "aardvark" and the last one "zebra", we will have 13 million vectors with the size of $(13,000,000 * 1)$.

$$w^{aardvark} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, w^{a} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, w^{at} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}, \ldots w^{zebra} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix}$$

It may seem that the problem here is the huge size of these vectors but apart from that, there are two more important problems to address:

- Having an extremely large vector with mostly zero values (this is called a sparse matrix)
- Assumption of semantic independence of tokens

Large vectors with mostly zero values in general have two issues: 1 - They won't easily fit into the memory thus they require a significant amount of memory 2- They have a high computational cost(time) since performing different operations requires intense computation.

On the other hand, the second general problem with One-Hot Encoding in this specific case is the principal assumption of it. While it is applicable for perfectly independent entities (as we saw in our example of geometric objects) it won't give us useful info about entities like words which are far from being independent. Using One-Hot Encoding we are assuming that for example words "cat" and "kitten" do have not any shared features whatsoever! (their dot product will be zero). One solution to overcome such a problem is to use an alternative vector- a dense vector- which is considerably shorter than the original vector we were dealing with. Before going any further, let's make sure that we agree on what this dimension reduction means on a semantic level since while the computational advantage is obvious, reducing features would be less clear. To explain this dimension reduction let's use a similar (but not equal) example to respect the problem we have in hand. Imagine that you need to create a vector representation of the following 10 colours:

$$black, white, red, lime, blue, yellow, cyan, magenta, silver, gray$$

first, let's take a look at One-Hot Encoding:

$$\begin{array}{c} \\ b \\ w \\ r \\ l \\ bl \\ y \\ c \\ m \\ s \\ g \end{array} \begin{array}{c} \begin{array}{cccccccccc} b & w & r & l & bl & y & c & m & s & g \end{array} \\ \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \end{array}$$

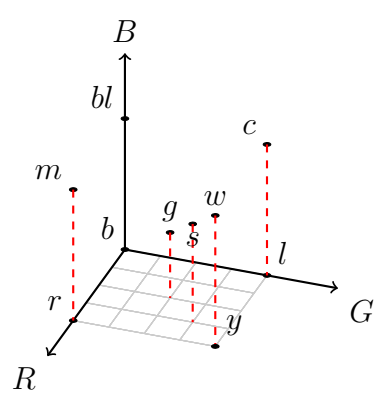
As we have mentioned before, the above representation is perfectly able to address each color so if you give me a vector like this:

$$V = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Having this vector (a localist presentation) I can tell you that you are talking about the "blue" colour, which means we have successfully created a vector that can take us back to the symbol of colour. What I can't tell you though, is for example its similarity or dissimilarity to other colors(relationship among words), but it doesn't come as a surprise(I hope!) as we already knew that One-Hot Encoding while capable of creating a vector representation, is not able to preserve the syntactic (structure) and semantic (meaning) relationships among entities. Now let's try a different representation (embedding) of these words. Following you see the result of using the RGB colour model:

$$\begin{array}{c} \\ b \\ w \\ r \\ l \\ bl \\ y \\ c \\ m \\ s \\ g \end{array} \begin{array}{c} \begin{array}{ccc} R & G & B \end{array} \\ \begin{pmatrix} 0 & 0 & 0 \\ 255 & 255 & 255 \\ 255 & 0 & 0 \\ 0 & 255 & 0 \\ 0 & 0 & 255 \\ 255 & 255 & 0 \\ 0 & 255 & 255 \\ 255 & 0 & 255 \\ 192 & 192 & 192 \\ 128 & 128 & 128 \end{pmatrix} \end{array}$$

Did you notice? we just went from a sparse to a dense matrix! Now instead of having 10 columns, we have just 3 columns R, G and B which show the intensity of each light in the specific colour.



Even using such a simple example, such an embedding can give us immediate information about entities and their relationship together. (Based on the 3D scatter plot above try to find out how different colours are related or how can we can get from one to another).

In the colour example, we were lucky enough to have already ready features like RGB lights. Unfortunately not always have the luxury of having representative features performing our embedding. What would be ideal is to have features that we can use to describe the words, something that can preserve both the syntactic and semantic characteristics of words.

# 3 Word2Vec

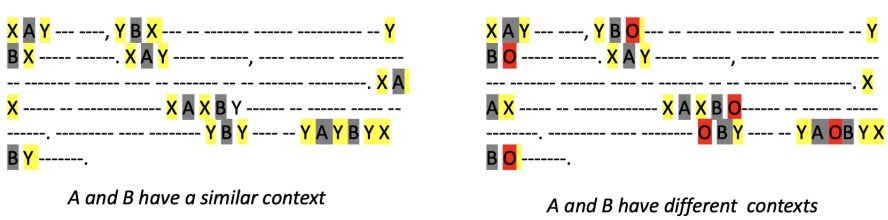
One of the latest (but not the most recent) prediction-based methods is the Word2Vec software package by Milkov and his team in Google in 2013. Word2Vec works based on a simple NLP idea called distributional similarity which proposes the idea that you can get a lot of value by representing the meaning of a word by looking at the context in which it appears (theory of meaning).

Before we dive into an example which helps us understand how Word2Vec works under the hood, let's revisit an important concept about language and meaning, famously summed up by J.R. Firth: "You shall know a word by the company it keeps." This principle suggests that a word's meaning is closely tied to the words that surround it.

So, how do we use this idea to understand the relationships between words and their contexts? This is where Word2Vec shines.

Let's look at an example. Imagine a sentence with words $A$ and $B$, highlighted in grey in our text. We're interested in the words that appear right before and after $A$ and $B$. If these surrounding words are similar—say both are surrounded by words $X$ and $Y$—then, according to Firth's principle, $A$ and $B$ are likely to have related meanings because they share similar contexts.

On the other hand, if the words surrounding $A$ and $B$ are different, we might conclude that their meanings are not as closely related. This is the insight that Word2Vec uses to learn about word meanings from large amounts of text.




We have just seen an extremely simplified version of the Word2Vec algorithm. Let's see what are the main differences between our example and the actual algorithm:

- **length of text**: instead of dealing with a short document as in the example, Word2Vec usually performs on a large corpus of data.
- **context**: unlike what we saw in the example, the actual example doesn't consider two words immediately before and after the target word. Instead, it considers a fixed

window with a predefined length of preceding and following words with respect to the target word.

- **scope**: While in our simple example, our goal was to check the similarity between two specific words, implementing the Word2Vec algorithm, we created a vector representation for all tokens present in the corpus. These vectors later give us the possibility to perform further analysis like similarity and transformation as we will see later in detail.

- **algorithm**: In the example we compared the context of target words and based on the combination of context words we have decided that $A$ and $B$ are similar in the image on the left and not similar in the image on the right. Word2Vec on the other hand uses a different approach which is predicting the context from the target and vice versa. Doing so it can understand which words have a similar context. (***CBOW*** and ***Skip-Gram algorithms***)

## 3.1 Training

In our example, we have replicated a very simple version of Word2Vec training methods. The actual algorithm uses - among others- the following techniques:

**Negative Sampling** Negative sampling is a technique that makes the training process more efficient. Normally, when training the Word2Vec model, for each word you would need to update the weights for all words in the vocabulary—this can be very slow if you have a large vocabulary.

Instead of updating all words, with negative sampling, you only update a small number of 'negative' words (words that are not the current focus word) along with the 'positive' word (the current focus word).

Here's how it works in simple terms: For each word that the model is trying to learn (the positive word), it also picks a small number of random words (the negative words) to update weights for. The idea is to make the model better at telling the positive word apart from these random negative words, rather than updating the weights for all words in the vocabulary. This way, the model learns the same general idea but with much less computation.

**hierarchical Softmax** Hierarchical softmax is another technique used to speed up the training process, especially when the vocabulary is large. Instead of computing probabilities for every word in the vocabulary when predicting a word, hierarchical softmax uses a binary tree representation of the vocabulary where each word is a leaf on the tree.

Imagine you have to pick a fruit from a large basket without looking. You could search through the entire basket to find the one you want, or you could organize the fruits into smaller groups (like berries, citrus, etc.) and then make a series of simple decisions to narrow down your search.

Hierarchical softmax does something similar with words. It organizes them in a tree, and each prediction involves making a series of decisions, going from the root of the tree down

to the specific word. This turns a potentially very large number of comparisons into just a handful, speeding up the process significantly.

So, instead of calculating the probability for each word against all other words in the vocabulary, hierarchical softmax calculates the probability of a sequence of decisions that lead to the target word. This reduces the computational complexity from being dependent on the number of words in the vocabulary to being dependent on the depth of the tree, which is much smaller.

## 3.2 Mechanism

Now that we have seen a simplified example and its main differences with Word2Vec, let's see in more detail how Word2Vec works

we need to have a large amount of textual data. It's important to notice these data are not a collection of individual words but a big set of sentences (corpus). Assuming that our corpus has $T$ words, we can address each word like this:

$$t = 1, 2, ..., T$$

Next, we assume that each word in this limited vocabulary is re-presentable by a vector and to start we assign each word a random vector. Iterating through all corpus, we pick each word (centre word $w_j$) and assume that it (its representation in this case) should be able to predict its context. The context of each word is the words in a window with a length of $m$ around it. We repeat these iterations hundreds of thousands of times, each time changing the representation in a way that the prediction will be better. We can summarize what we have said until now in the following formula which calculates the likelihood of our model:

$$L(\theta) = \prod_{t=1}^{T} \prod_{-m \leqslant j \leqslant m} P(w_{t+j} \mid w_t; \theta)$$

$$j \neq 0$$

$\boldsymbol{L}$: Model Likelihood or how good is our model in predicting context from centre words
$\boldsymbol{\theta}$: Model variables we need to optimize. In our case, the only variable we have in hand is the vector representations of words. It means the only thing that defines the performance of our probability model is the vectors we come up with for each word in the corpus.
$\boldsymbol{m}$: The window size for the context words around the centre word
$\boldsymbol{P(w_{t+j} \mid w_t; \theta)}$: Probability of having $w_{t+j}$ as a context word given the $w_t$ as center word and $\theta$ as vector representation.

While being able to calculate the model performance (model likelihood) for a certain $\theta$ is good, we need something that helps us with actually finding the best $\theta$. To do so we use a certain function called "objective function"(or loss function) which is a transformation of model likelihood. Doing such a transformation we will have $J(\theta)$ which we can look at as *Error* we get, assigning vectors to words. It means by minimizing it we are getting the best

vectors possible for each word. Let's take a look at this transformation and examine the internal pieces:

$$J(\theta) = -\frac{1}{T}\log L(\theta) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{-m\leqslant j\leqslant m}\log P(w_{t+j} \mid w_t;\theta)$$

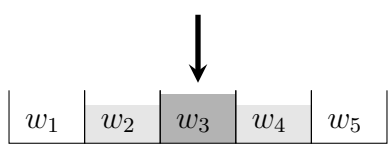
$$j \neq 0$$

As we said before $J(\theta)$ is the transformed form of $L(\theta)$. Let's see what are the differences:

- Negative sign(-): indicates that we want to minimize the objective function. (we can achieve the same result also with maximizing the objective function)
- $\frac{1}{T}$: By adding it we calculate the Likelihood for every word in a corpus of $T$ words.
- log: using logarithm (a usual operator in optimization processes) we convert $\prod$ to $\sum$

Before talking about the last piece, $P(w_{t+j} \mid w_t;\theta)$, let's summarise what we said until this point: Our final goal is to find vectors that can understand the meaning behind words and can discover the similarities between them. To find such vectors we have introduced a loss function which by minimizing it we achieve our primary goal. Minimizing this cost function means maximizing the total probability that our model can predict the correct context from each word.
Let's see a simple example: Imagine that we are dealing with a corpus with only 5 words and let's consider the window size as 1. so:

$$T = 5 \;\; and \;\; m = 1$$




So we can expand the loss function as:

$$\begin{aligned}
L(\theta_i) &= -\frac{1}{5}\sum_{t=1}^{5}\sum_{-1\leqslant j\leqslant 1}\log P(w_{t+j} \mid w_t) \\
&= -\frac{1}{5}(\log\ P(w_2 \mid w_1)+ \\
&\log P(w_1 \mid w_2) + \log P(w_3 \mid w_2)+ \\
&\log P(w_2 \mid w_3) + \log P(w_4 \mid w_3)+ \\
&\log P(w_3 \mid w_4) + \log P(w_5 \mid w_4)+ \\
&\log P(w_4 \mid w_5))
\end{aligned}$$

OK, now let's talk about the elephant in the room! $P(w_{t+j} \mid w_t;\theta)$. We said before that it is the probability of having a context word given the centre word. But how can we calculate

such a probability? As can be seen, we are dealing with a conditional probability or more precisely, a probability distribution. Take another look at the result of the previous example. The last 5 lines show for each word of 5 words - what is the probability of having such a word given its context words ( lines one and five have just one expression since the only context of words 1 and 5 are respectively words 2 and 4). To calculate this probability, we can use a trick: Considering two different vector representations for each word based on its position, showing it with $v$ when it's a centre word and mentioning it with $u$ when it's a context word. Having such an assumption we can use the following formula:

$$P(o \mid c) = \frac{e^{u_o^T v_c}}{\sum_{w \in V} e^{u_w^T v_c}}$$

$\boldsymbol{u_o}$: vector of context word
$\boldsymbol{v_c}$: vector of center word
$\boldsymbol{u_w}$: vector of a word not in a center position
$\boldsymbol{P(o \mid c)}$: Probability of having Outside word $\boldsymbol{o}$ having center word $\boldsymbol{c}$

What we see in this formula is a combination of two things:

- Dot product of two vectors
- Softmax function

It means we can re-write our formula in the following way:

$$P(o \mid c) = Softmax(Probability(\mathbf{o}\ being\ context\ for\ \mathbf{c})) = Softmax(dot\,product(o.c))$$

### 3.2.1 Dot-product

As we said before, a vector has two values. The dot product of two vectors (a scaler) reflects both the magnitude and directions of both vectors:

$$a.b = |a| \times |b| \times \cos(\theta)$$

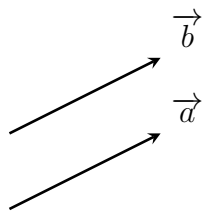
$\boldsymbol{|a|}$ **,** $\boldsymbol{|b|}$: length(magnitude) of $a$ and $b$ vectors.
$\boldsymbol{\theta}$: Angle between directions of $a$ and $b$ vectors.

Looking at the formula above it is quite easy to understand how the magnitude of vectors contribute to the outcome, the role of $\cos(\theta)$ might be less evident. We know that $\cos(\theta)$ should somehow reflect the angle between two vectors but there are two questions here:

- Why do we need to use angle in the first place? why using vector lengths is not enough?
- How exactly $\cos(\theta)$ represents the angle? let's say, why not we use $sin(\theta)$ instead?

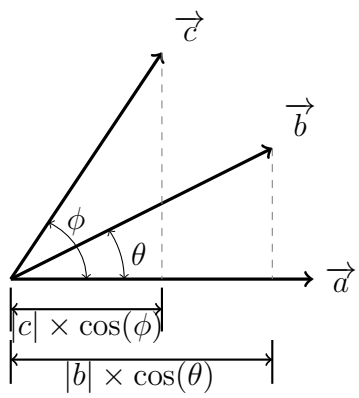
To answer the first question, let's first remind ourselves of the goal of a dot-product: combining two vectors to obtain their product. So we can expect that having vectors as ingredients,

we need to consider all their characteristics which as we mentioned before, are magnitude and direction. for example, looking at the following simple example we can see that it's trivial to think that given that both vectors have the same directions it's possible to say that we can obtain the multiplication of these vectors by multiplying their lengths:

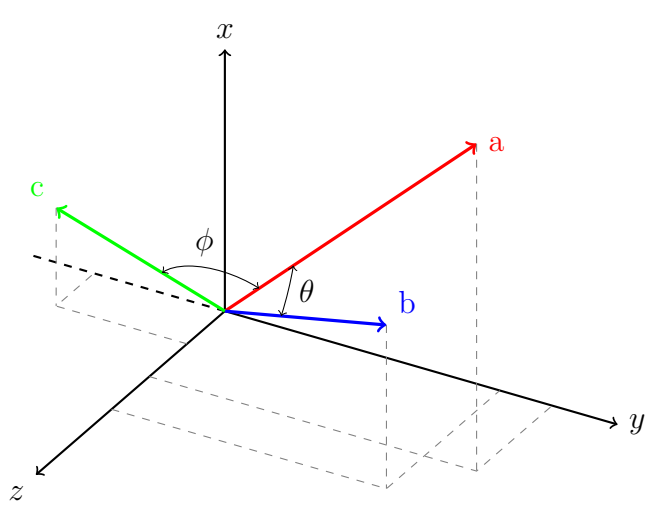



$$|a| \times |b|$$

But the things won't remain always as simple as we have seen above. Look at this example:




The three vectors $\overrightarrow{a}$, $\overrightarrow{b}$ and $\overrightarrow{c}$ have the same length ($|a| = |b| = |c|$) but we to calculate $a.b$ or $a.c$ we can't just simply multiply their lengths because we can just do such a thing only if two vectors have the same direction. So, we need to somehow convert one of the vectors in a way to has the same direction as the other vector. The way we can achieve this is to use the trigonometry *Cosine* function. As you can see in the figure above, after converting $\overrightarrow{b}$ and $\overrightarrow{c}$ using the *Cosine* function we can see that $b.a$ will be greater than $c.a$.



The situation we face dealing with words will be quite different from the example above in 2-dimensional space. As we said before, even if we are going to use dense vectors instead of sparse outcomes of one-hot encoding, the vector space still would be too complex to be visually demonstrated. In the figure above we see a 3-dimensional vector space with 3 vectors. Even without calculating dot products, we can almost certainly say that $a.b$ is greater than $a.c$ given that $\phi$ the angle between $a$ and $b$ is close to 90° which brings $c.b$ close to zero. As a result, we can see that in our case of word vectors, we can expect that similar vectors have a greater dot product with respect to the dissimilar vectors. Back to our formula, we can see that we have two parts indicating a dot product: $u_o^T v_c$ and $u_w^T v_c$. As you can see we are using matrix transpose since we can't directly multiply two columns vectors. Following is the general form of dot product:

$$A^T B = \begin{bmatrix} A_1, A_2, \ldots, A_n \end{bmatrix} \begin{bmatrix} B_1 \\ B_2 \\ \vdots \\ B_n \end{bmatrix} = A_1 B_1 + A_2 B_2 + \vdots + A_n B_n = A.B$$

### 3.2.2 Softmax Function

Before, explaining the softmax function let's take a quick look at the equation we have started with: (Remember that its output should be a

$$\frac{e^{\boldsymbol{u_o^T v_c}}}{\sum_{w \in V} e^{\boldsymbol{u_w^T v_c}}}$$

The blue parts are the pairwise dot-product of vectors as we have discussed in the previous part. Now we should explore the rest of the equation. What we see in the above equation is the implementation of dot-products in the following function:

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \qquad for\ i = 1, \ldots, K$$

What we can infer from this function is that it's calculating some sort of proportion of the total sum, something like this:

$$\frac{Something}{Sum(Everything)}$$

The first thing that comes to mind is to use it to understand how the dot-product of a certain word and a centre word is different from the dot-product of that word with other words of the corpus. Something like this:

$$\frac{Probability(A\,certain\,word\,being\,context\,of\,a\,specific\,word)}{Sum(Probability(A\,certain\,word\,being\,context\,of\,any\,other\,word\,in\,the\,corpus))}$$

So maybe we can write an equation similar to this:

$$\frac{u_o^T v_c}{\sum_{w \in V} u_w^T v_c}$$

The above equation gives us a vector with a length of $V$ (our corpus) in which each element is the following proportion:

$$\frac{dot - product(A\,certain\,word\,being\,context\,of\,a\,specific\,word)}{Sum(dot - product(A\,certain\,word\,being\,context\,of\,any\,other\,word\,in\,the\,corpus))}$$

Sounds great! There's just a small problem! Going back to the initial equation of likelihood, we should have had a probability part but What we get from the formula above is not a probability at all! There are two problems:

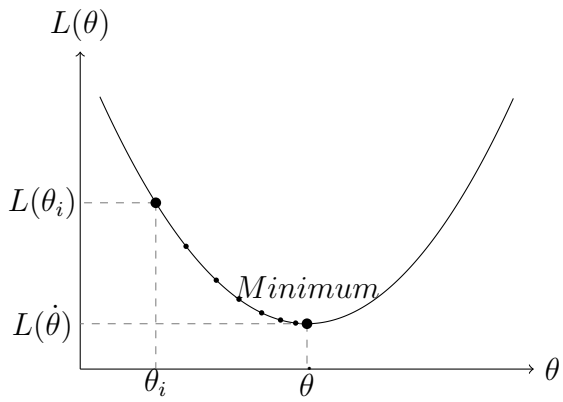
- Sum of the items in the vector won't be equal to 1.
- Each item is not necessarily between 0 and 1

The reason is quite simple! The outcome of a dot-product is a "scaler", not a probability ($0 \leqslant x \leqslant 1$). So we need to somehow "transform" these scalers to probabilities. This is the reason in the initial formula we used **e** to transform the scaler result of the dot products to a number between 0 and 1.

Ok, Until here, we saw all the pieces of Cost function one by one. or the last time let's summarize what we said until now and put these separate pieces together:

Cost function should be minimized $\overset{so}{\Rightarrow}$
Individual probabilities should be maximized $\overset{so}{\Rightarrow}$
Word vectors should represent the relationship between words

It means the only way we can minimize the cost function is to come up with a high probability for a certain word being a context for another certain word, which in turn, means we should somehow find the correct vectors for words but the question is: How can we find such vectors? The answer - at least from a conceptual point of view - is easy: Initiate each word with a random vector, calculate the Cost Function, change the vectors in a way that gives a lower cost, calculate the cost and repeat these steps until the cost can't go lower (you reach the global minimum for the cost). The following figure shows a simplistic representation of these steps: We start with a random set of vectors ($\theta_i$) and use it to calculate the cost ($L(\theta_i)$). Using a method called **gradient descent** (more on this later) we change the $\theta$ - little by little - until we get to the $\dot{\theta}$ which returns the minimum cost possible ($L(\dot{\theta})$).




We see that using gradient descent, eventually we can arrive at the point (word vectors) which minimizes the cost but how does gradient descent work? Simply put, it means taking small and directed(non-random) steps toward the minimum point from the starting point (which in our case was randomly generated). To take these steps we need to have two things:

- How big should be the step ($\alpha$)
- What should be the direction we move $\theta$ to

To understand these parameters, let's see the classic example of **GD**: Imagine that you're a climbing fan. You go to the bottom of a mountain, park your car and start climbing. after 6 hours, you're somewhere near the top of a mountain but since it's getting dark (and you're tired), you decide to go down back to where you've parked your car. The problem is, that there's a thick fog in the mountain and while you know your final destination is down (in our case, down means minimizing our cost function), you can't see the whole way until the destination point. So in this case you decide to choose a direction that seems right to you, take some steps, evaluate your position and choose another direction. Eventually, by repeating these actions, you'll(hopefully) get to your car. This is all **GD** does. Starting from a given initial point, it first calculates the partial depravities(A partial derivative shows you the direction of the point that makes the function minimum or maximum) then, using the step ($\alpha$) you've defined beforehand, it updates the  and recalculates the cost. Repeating these steps we'll finish with the best set of vectors for our corpus.
Now that we are familiar with the internal mechanism Word2Vec uses to calculate the word vectors, let's take a quick look at more detail on the tool it uses for such vectors.

As you can remember we have said that Word2Vec "assigns random vectors to words and keeps modifying them until they produce the minimum cost possible". To do this procedure, Word2Vec uses Neural Networks. Neural Networks are one of the most used algorithms for a wide range of applications from forecasting stock prices to adding cat ears to your selfies. Here we won't go into detail of NN since doing so needs its book(books?)  and will take us away from our goal but to have a general idea about NN let's take a look at it. Before jumping directly to NNs, let's start with a simple example: Snake game. In this game, you should guide the snake to the point having three rules: + You can choose between Horizontal and vertical moves but not both at the same time + The snake should hit himself!

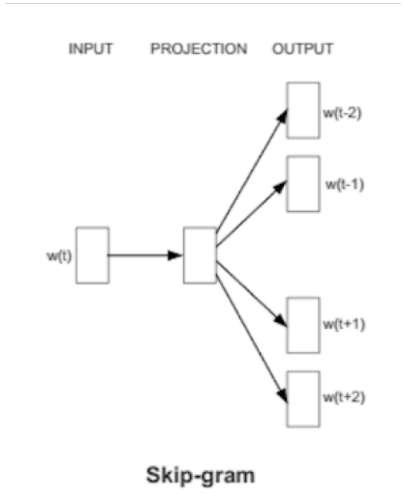
Now imagine a slightly different game:

- You can move Horizontally, Vertically, and Diagonally.
- You can jump, disappear and re-appear
- You can mix the previous two moves
- There are fixed and moving obstacles

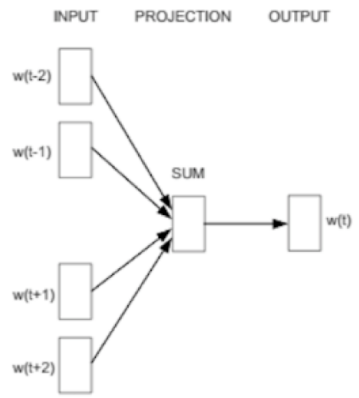
As you can see the simple Snake game suddenly becomes much harder as you should control more parameters , face harsher obstacles and achieve a harder goal. Now for the last time let's consider a different scenario:

- We have 1,000,000 parameters to control
- You can combine all these parameters and each combination has a different result
- Good luck!

Well, this time it seems we need something more than our fantastic human brains! That's where Neural Networks come to help us. What a Neural Network does is take hundreds or thousands (depending on the problem in hand) of solved cases and try different parameters, find out the set of them which produces the least amount of error(or is closer to the correct cases.) This is what happens inside the word2vec algorithm; as we gave it our corpus it considers the position of words as the correct neighbouring for the words. As a result, using Neural networks it returns word vectors which represent this neighbouring.
Until now, in all examples we have used a specific case: predicting context words having the centre word. This, called skip-gram- is one of the two internal algorithms Word2Vec can use. The other algorithm is called Continuous Bag Of Words (CBOW) which is when we try to predict the centre word using the context words. While having a similar goal, these algorithms return slightly different vectors for words.
Skip-gram: The input to the model is $w_i$, and the output is $w_{i-2}, w_{i-1}, w_{i+1}, w_{i+2}$. So the task here is **"predicting the context given a word"**.



CBOW: The input to the model is 2, 1, +1, +2, the preceding and following words of the target word. The output of the neural network will be. so the task is "predicting the word given its context" Skip-gram: The input to the model is, and the output is 2, 1, +1, +2. So the task here is **"predicting the context given a word"**.




CBOW

*Note: As a hyper-parameter, you should define the Max window size but don't forget that the algorithm won't necessarily use all the words in the window but instead it randomly chooses some of the words based on the distance they have from the target word (in a way that closer words have a higher chance to be picked)* According to the author of Word2Vec:

- Skip-gram: works well with a small amount of the training data, and represents well even rare words or phrases.
- CBOW: several times faster to train than the skip-gram, slightly better accuracy for the frequent words.

So it depends on your data and computational power

## 3.3 Implementing Word2Vec in Python and R

Implementing the Word2Vec algorithm can be approached differently in Python and R, with each language offering its own set of libraries and tools suited for this task.

### 3.3.1 Python Implementation

In Python, the **gensim** library is a robust and efficient library for Word2Vec implementation. It is highly optimized and allows for easy training, saving, and loading of Word2Vec models. Users can simply install gensim using pip and then utilize the Word2Vec class provided within the library.

Challenges in Python may include managing the preprocessing of text data, as gensim requires a list of tokenized sentences for training. Additionally, choosing the right set of hyperparameters such as vector size, window size, and the number of workers for parallel processing can significantly affect the quality of the resulting word vectors.

### 3.3.2 R Implementation

In R, the **text2vec** package is a popular choice for working with Word2Vec. It is designed to be memory and computationally-efficient. However, the R ecosystem may not be as rich as Python's for NLP tasks, and the text2vec package might have fewer features compared to gensim.

A key challenge in R is the typically steeper learning curve for efficient text data handling and manipulation. Users must also be cautious with memory management, as R can be less forgiving than Python in terms of memory usage.

### 3.3.3 General Considerations

Regardless of the programming language, implementing Word2Vec comes with several common challenges. One must have a good understanding of the underlying algorithm to select appropriate hyperparameters. Adequate preprocessing of the text data is crucial, and this often involves tokenization, removal of stop words, and sometimes lemmatization or stemming. Moreover, training Word2Vec models on large datasets can be time-consuming and may require significant computational resources.